Lab Handout 4-B

Preprocessing Pipeline, Feature Selection, and Label-Leakage Prevention

Capstone — Module 4: AI-Driven Classification Pipeline

1 Objective

Convert the validated dataset from LH4-A into model-ready matrices: drop join-mid-stream artifact rows, encode labels, select task-specific feature sets that avoid label leakage, and fit a `StandardScaler` that downstream classifiers will reuse. The deliverable is a reusable `Preprocessor` class and saved artifacts (`X_train.npy`, `y_train.npy`, `preprocessor.pkl`) for all four classification tasks (SF, modulation, emitter ID, packet type).

2 Prerequisites

- Lab Handout 4-A complete (`validation_manifest.json` present in `Code/AIML/`).
- Python with `scikit-learn` and `joblib` installed.

3 Background: Why Four Feature Sets, Not One?

Module 4 trains four classifiers: spreading-factor detection, modulation classification, emitter identification, and packet-type classification. Each has a different list of columns that are *forbidden* because they would trivially reveal the label:

Task	Label	Columns Forbidden (Would Leak Label)
SF Detection	`label_sf`	`spread_factor` (self-reported copy), `freq_error_hz` (perfectly session-confounded with SF in the current dataset, $r = 1.000$ per LH3-H EDA 3)
Modulation	`label_mod`, `mod_kind_label`	`mod_kind` (self-reported copy), `spread_factor` (would be 0 for non-LoRa modulations once C3 is collected)
Emitter ID	`label_beacon`	`beacon_id` (self-reported copy of the label)
Packet Type	`label_pkt`	`pkt_type` (*is* the label — extracted from the low 2 bits of the `Modulation Code` byte by LH2-F)

Table 1: Label-leakage table. Any of these forbidden columns left in the feature matrix will push test accuracy to ∼100% — not because the classifier works, but because it is reading the answer.

The Silent Leak

Label leakage rarely produces an error message. It produces an impressive accuracy number. The only way to catch it is to enumerate your forbidden columns *before* training, and the only way to keep that list current is to derive it mechanically from the data dictionary. If you add a new feature to the CSV later, revisit this table.

The `freq_error_hz` case is special: it is not a label by construction, but in this dataset every SF was collected in a separate session and the SX1276 crystal drifted ∼131 Hz between sessions. The LH3-H EDA pinned the SF/freq_error correlation at $r = 1.000$. A classifier trained with that column would just learn "which session," not "which SF." Until the dataset is rebuilt with interleaved-SF sessions, the column is forbidden for the SF task.

4 Procedure

4.1 Step 1 — Drop sentinel rows

Rows with `obs_count == 1` have undefined `inter_arrival_ms`, `rssi_delta`, and `priority_score` (nothing to compare the first observation against). Drop them globally before any feature selection.

```
import pandas as pd
df = pd.read_csv("merged_dataset.csv")
n0 = len(df)
df = df[df["obs_count"] > 1].reset_index(drop=True)
print(f"Dropped {n0 - len(df)} sentinel rows ({(n0 - len(df))/n0:.1%})")
```

4.2 Step 2 — Build the `Preprocessor` class

```
import joblib
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from pathlib import Path

FEATURE_COLS_BASE = [
    "rssi", "snr", "inter_arrival_ms", "rssi_delta",
    "obs_count", "priority_score", "distance_m",
]

FORBIDDEN = {
    "sf":     ["spread_factor"],
    "mod":    ["mod_kind", "spread_factor"],
    "beacon": ["beacon_id"],
}

LABEL_COL = {
    "sf":     "label_sf",
    "mod":    "label_mod",
    "beacon": "label_beacon",
}

class Preprocessor:
```

```python
    def __init__(self, task):
        assert task in FORBIDDEN, f"Unknown task: {task}"
        self.task = task
        self.feature_cols = [c for c in FEATURE_COLS_BASE
                             if c not in FORBIDDEN[task]]
        self.scaler = StandardScaler()
        self.encoder = LabelEncoder()

    def fit_transform(self, df):
        tr = df[df["split"] == "train"]
        X_tr = tr[self.feature_cols].to_numpy(dtype=np.float32)
        y_tr = self.encoder.fit_transform(tr[LABEL_COL[self.task]])
        X_tr = self.scaler.fit_transform(X_tr)
        return X_tr, y_tr

    def transform(self, df, split):
        sp = df[df["split"] == split]
        X = sp[self.feature_cols].to_numpy(dtype=np.float32)
        y = self.encoder.transform(sp[LABEL_COL[self.task]])
        X = self.scaler.transform(X)
        return X, y

    def save(self, out_dir):
        out = Path(out_dir)
        out.mkdir(parents=True, exist_ok=True)
        joblib.dump({"scaler": self.scaler,
                     "encoder": self.encoder,
                     "feature_cols": self.feature_cols,
                     "task": self.task}, out / "preprocessor.pkl")
```

Listing 1: `preprocess.py` — the reusable preprocessing class used by every Module 4 classifier.

4.3 Step 3 — Generate task-specific matrices

```python
from preprocess import Preprocessor
import numpy as np

for task in ["sf", "mod", "beacon"]:
    pre = Preprocessor(task)
    X_tr, y_tr = pre.fit_transform(df)
    X_va, y_va = pre.transform(df, "val")
    X_te, y_te = pre.transform(df, "test")

    print(f"\n=== Task: {task} ===")
    print(f"Features: {pre.feature_cols}")
    print(f"Classes:  {list(pre.encoder.classes_)}")
    print(f"Shapes:   X_tr={X_tr.shape}, X_va={X_va.shape}, X_te={X_te.
        shape}")

    out = f"artifacts/{task}"
    pre.save(out)
    np.save(f"{out}/X_train.npy", X_tr); np.save(f"{out}/y_train.npy",
        y_tr)
```

```
    np.save(f"{out}/X_val.npy",   X_va); np.save(f"{out}/y_val.npy",
        y_va)
    np.save(f"{out}/X_test.npy",  X_te); np.save(f"{out}/y_test.npy",
        y_te)
```

Listing 2: Run preprocessing for all four tasks.

4.4 Step 4 — Verify the scaler

A fitted `StandardScaler` on the training set produces zero mean and unit variance per feature:

```
for task in ["sf", "mod", "beacon"]:
    X_tr = np.load(f"artifacts/{task}/X_train.npy")
    mu, sd = X_tr.mean(axis=0), X_tr.std(axis=0)
    print(f"{task}: mean max|mu|={np.abs(mu).max():.3e}, "
          f"std range=[{sd.min():.3f}, {sd.max():.3f}]")
    assert np.abs(mu).max() < 1e-5
    assert np.all(np.abs(sd - 1) < 1e-5)
```

Validation and test matrices will *not* have exactly zero-mean/unit-variance because the scaler was fit on training data only — that is correct behaviour and the whole reason `fit_transform` is called on train while plain `transform` is called on val and test.

When Not to Scale

Tree-based classifiers (decision tree, random forest, extra-trees) are invariant to monotonic transformations of single features. Scaling them does not hurt but adds needless overhead. LH4-C and LH4-D still load the scaled matrices because using a single set of artifacts keeps the experiment repeatable across every model, but do *not* quote a decision-tree result as "better with scaling" — it is identical up to floating-point noise.

4.5 Step 5 — Leakage sanity check

Train a quick logistic regression on each task using *only* the forbidden columns, and confirm accuracy is near 100%. This demonstrates that the forbidden list is correct: leaving those columns in would trivialise the problem.

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

for task, bad in FORBIDDEN.items():
    tr = df[df["split"] == "train"]
    te = df[df["split"] == "test"]
    # label_mod is a string; encode it the same way the Preprocessor did
    from sklearn.preprocessing import LabelEncoder
    enc = LabelEncoder()
    y_tr = enc.fit_transform(tr[LABEL_COL[task]])
    y_te = enc.transform(te[LABEL_COL[task]])
    X_tr = tr[bad].to_numpy(dtype=float)
    X_te = te[bad].to_numpy(dtype=float)
    sc = StandardScaler().fit(X_tr)
    clf = LogisticRegression(max_iter=1000).fit(sc.transform(X_tr), y_tr)
    acc = clf.score(sc.transform(X_te), y_te)
```

```
    print(f"{task} (forbidden-only features): test acc = {acc:.3f}")
```

A result below ∼0.95 means the forbidden list is incomplete for that task; revisit and add the missing column.

5 Verification Checklist

- ☐ Sentinel rows dropped; count matches LH4-A audit summary.
- ☐ `Preprocessor` class saves and reloads without error.
- ☐ All four tasks have six saved arrays (`X_train`, `X_val`, `X_test` and three matching `y_`).
- ☐ Training-set scaler produces zero mean / unit variance within 10^{-5}.
- ☐ Leakage sanity check: each task's forbidden-only classifier scores ≥0.95 (including the packet-type task, where the `pkt_type`-only classifier should score 1.00 trivially because the feature *is* the label).

Lab Handout 4-B Deliverable

1. `preprocess.py`: the reusable `Preprocessor` class.
2. `artifacts/{sf,mod,beacon,pkt}/` directories, each containing `preprocessor.pkl` and six `.npy` matrices.
3. Leakage sanity-check output showing ≥0.95 test accuracy for each task when only forbidden columns are used.
4. Feature-list manifest: one page per task (four pages total) listing (a) the chosen feature columns, (b) the forbidden columns, and (c) a 1-sentence physical justification for why each forbidden column leaks. The packet-type page should explicitly call out that `pkt_type` is itself the label (extracted from the low 2 bits of the radio packet's `Modulation Code` byte).